M03: The Map: Navigating Linux

> "A config file was changed last night and the app is broken. You need to find it — but you don't know where to start."

The Scene

- Monday morning — the deployment dashboard is **red**
- Someone on the weekend team changed a config file
- Nobody remembers **which file** or **where**
- Thousands of directories, tens of thousands of files
- No search bar. No file browser. Just a **blinking cursor**

> To solve this, you need two things: the ability to move around the filesystem, and the knowledge of where things are supposed to live. That's this module.

Where Am I? — `pwd`

- `pwd` = **Print Working Directory** — shows your full location
- No flags, no arguments — just type `pwd`
- Returns an **absolute path** starting from root (`/`)

```
$ pwd
/home/anas
```

- Your prompt shows an abbreviated path — `pwd` gives the **complete, unambiguous** answer

> You're hunting a broken config. Step one: know where you're standing.

What's Here? — `ls`

- `ls` = **list** — shows files and directories at your location
- `ls -lah` is your **go-to combo** — long format, all files, human-readable sizes

```
$ ls -lah
drwxr-xr-x  2 anas anas 4.0K Jan 12 11:15 configs
-rw-r--r--  1 anas anas 1.2K Jan 15 09:30 deploy.sh
drwxr-xr-x  3 anas anas 4.0K Jan 13 08:45 logs
```

Flag	What It Does
`-l`	**Long format** — permissions, owner, size, date
`-a`	**All** — includes hidden files (starting with `.`)
`-h`	**Human-readable** — sizes as K, M, G
`-t`	**Time-sorted** — newest files first

> `ls -lah` will become muscle memory. You'll type it hundreds of times.

Moving Around — `cd`

`cd` = **Change Directory** — moves you to a new location

Command	What It Does
`cd /var/log`	Go to an **absolute** path (starts from `/`)
`cd configs`	Go to a **relative** path (from where you are)
`cd ..`	Go **up one level** (parent directory)
`cd` or `cd ~`	Go **home** (the `$HOME` you learned in M02)

```
$ cd /var/log
$ pwd
/var/log
$ cd -
/home/anas
```

> `cd -` is your "back button" — toggles between your last two locations. When you're jumping between configs and logs during an investigation, this saves you from retyping long paths.

Absolute vs. Relative Paths

- **Absolute** — starts with `/`, works from anywhere, like a full street address
- **Relative** — starts from your current directory, like "two doors down"

Symbol	Name	Meaning
`/`	Root	The very top of the filesystem
`.`	Dot	Current directory
`..`	Double dot	Parent directory (one level up)
`~`	Tilde	Your home directory (`$HOME`)

- **Scripts and configs** — use absolute paths (always reliable)
- **Interactive work** — relative paths are faster to type

> When in doubt, use absolute paths. A relative path that works in `/home/anas` might point to the wrong file from `/var`.

Now You Can Navigate — But Where Should You Go?

You can move around the filesystem. But **where do configs live? Where do logs go? Where do installed programs end up?**

On Windows: different versions put things in different places. Registry. `Program Files`. `Program Files (x86)`. No clear pattern.

Linux is different:

- One starting point: `/` — every file, device, and drive lives under it
- The structure is **standardized** — Ubuntu, Amazon Linux, Alpine, CentOS all follow the same map
  `/etc` is config on every distro. `/var/log` is logs everywhere. Learn it once.

> This map is why a bash script written for Ubuntu runs on a Google Cloud VM running CentOS without modification. Same structure, everywhere.

The Complete Map — Part 1: Your Files, Config, and Programs

Directory	What It Holds
`/home`	**User home directories** — `/home/anas`, `/home/sara` — personal files, SSH keys, shell configs
`/root`	**Root user's home** — the superuser's separate home directory
`/etc`	**System configuration** — every service, every setting — nginx.conf, hostname, network setup
`/tmp`	**Temporary files** — cleared on reboot; never store anything important here

Directory	What It Holds
`/bin`	**Essential commands** — ls, cp, mv — the basics the system needs to boot
`/usr/bin`	**Installed commands** — python3, git, node — the full toolbox
`/usr/share`	**Docs and assets** — man pages, icons, shared data
`/lib`	**Shared libraries** — the `.so` files programs depend on to run

The Complete Map — Part 2: Variable Data, Virtual, and Devices

Directory	What It Holds
`/var`	**Variable data** — logs (`/var/log`), web files (`/var/www`), databases (`/var/lib`)
`/opt`	**Optional third-party software** — CloudWatch Agent, Datadog, monitoring tools install here
`/mnt`	**Manual mount point** — temporary mounts for extra disks or network drives

Directory	What It Holds
`/proc`	**Virtual filesystem** — live kernel data (CPU, RAM, processes); not real files on disk
`/sys`	**Virtual hardware interface** — kernel view of devices and drivers; also virtual
`/dev`	**Device files** — hard drives (`/dev/sda`), terminals (`/dev/tty`), null (`/dev/null`) — in Linux, everything is a file, including hardware

> `/proc` and `/dev` are not real directories with files on disk. They are **interfaces** — the kernel generates their content live, on demand. When you read `/proc/cpuinfo`, the kernel writes the answer in that moment.

The Five Directories That Matter Daily

You just saw 16 top-level directories. You do not need to memorize all of them.

For cloud work, you'll spend 90% of your time in five places:

`/etc`

config (nginx, hostname, network, every service)

`/var/log`

logs (first place to check when something breaks)

`/home`

your files (SSH keys, scripts, personal data)

`/opt`

third-party tools (CloudWatch Agent, Datadog)

`/proc`

live system info (CPU, memory, processes)

> The rest exist for a reason, and you'll pick them up naturally. But these five? Internalize them.

`/proc` — Live Data From the Kernel

`/proc` is a **virtual filesystem** — the kernel generates it live. Nothing in `/proc` exists on disk.

Key files:

Path	What It Tells You
`/proc/cpuinfo`	CPU model, cores, speed
`/proc/meminfo`	Total RAM, free RAM, swap
`/proc/uptime`	Seconds since last boot
`/proc/loadavg`	System load over 1, 5, 15 minutes

- Numbered directories (`/proc/1`, `/proc/245`) = one per running process
- **Cloud connection:** CloudWatch, Datadog, Prometheus all read from `/proc` — that CPU graph in your AWS console came from here

> To read the contents of these files you'll use `cat` — a command you'll learn next module. For now, just know where to look.

`/var` — Where Things Change

- `var` = **variable** — data that grows and changes during normal operation
- If `/etc` is where you **configure** the system, `/var` is where the system **records what it does**

The key subdirectories:

`/var/log`

logs that grow continuously (syslog, auth.log, app logs)

`/var/www`

web content served to visitors

`/var/lib`

persistent state data (databases, package manager state)

`/var/cache`

cached data you can clear in an emergency

> **Real-world warning:** `/var/log` is the number-one reason servers run out of disk space. Logs grow forever unless you set up log rotation. If you see "No space left on device," check `/var/log` first.

Finding Files — `find`

- `find` searches **recursively** through every subdirectory
- This is the command that **solves our mission**

```
$ find /etc -name "*.conf" -mtime -1
/etc/nginx/nginx.conf
```

Criteria	What It Matches
`-name "*.conf"`	Files matching a name pattern (quote the wildcards!)
`-type f` or `-type d`	Only files or only directories
`-mtime -7`	Modified within the last 7 days
`-size +100M`	Larger than 100 megabytes

> That broken config? `find /etc -name "*.conf" -mtime -1` narrows thousands of files to the handful that changed last night. Investigation: started.

Wildcards: Matching Multiple Files

- The shell expands **wildcard patterns** into matching filenames before the command runs

Pattern	What It Matches	Example
`*`	Any characters (including none)	`*.log` matches all log files
`?`	Exactly one character	`app.??g` matches `app.log`, `app.cfg`
`[abc]`	One character from the set	`file[123].txt`

```
$ ls *.conf
app.conf  database.conf  network.conf
```

> Remember: `*` matches everything. **Always run `ls` with your pattern first** before using it with a destructive command like `rm`.

Locating Commands — `which` and `type`

- `which` — shows the full path to a command's executable
- `type` — tells you if a command is a file, a shell builtin, or an alias

```
$ which find
/usr/bin/find

$ type cd
cd is a shell builtin
```

- `which` answers: **"Is this tool installed?"** — no output means it's not in your `$PATH` (from M02)
- `type` answers: **"What exactly is this command?"** — catches hidden aliases

> When a command behaves unexpectedly, `type` reveals if an alias is adding flags you didn't ask for.

Aha! Moment 💡

Think about what this means:

- SSH into an AWS EC2 instance → `/etc` has configs, `/var/log` has logs
- Docker container based on Alpine → same structure
- Google Cloud VM running CentOS → same structure
- Your Ubuntu VM right now → same structure

Real investigation scenarios:

→ "Which nginx config is active?" → `find /etc -name "nginx.conf"`

→ "What did the deployment script touch?" → `find /var/www -mtime -1`

→ "Is Python installed?" → `which python3`

→ "Why is the disk full?" → check `/var/log`

> The Linux filesystem is a **contract** between every distro and every admin. This isn't just organization — it's the reason DevOps automation works at all.

Quick Reference

Navigate:

Command	What It Does
`pwd`	Show full path to current location
`ls -lah`	List files with details, hidden files, human sizes
`cd /path`	Go to absolute path
`cd ..`	Go up one level
`cd -`	Toggle back to previous location

Find:

Command	What It Does
`find /etc -name "*.conf"`	Search for files by name pattern
`find /var -mtime -1`	Files modified in the last day
`which python3`	Show where a command lives
`type cd`	Show what a command really is

The 5 directories that matter:

```
/etc        → config
/var/log    → logs
/home       → your files
/opt        → third-party tools
/proc       → live system info
```

Mission Complete: The Map ✅

You can now navigate any Linux server with purpose.

- `pwd` / `ls` / `cd`
  know where you are, see what's there, move around
- **Absolute vs. relative paths**
  absolute starts with `/`, always reliable
- **Linux has one root (`/`)**
  one tree, every distro, every cloud
- **The five directories**
  `/etc`, `/var/log`, `/home`, `/opt`, `/proc` — internalize these
- **`/proc` and `/dev` are virtual interfaces**
  not real files on disk
- **`find` searches the entire filesystem at once**
  faster than any GUI

> Next mission: you can navigate to a file. Now you need to read it, copy it, move it, and delete it with confidence. That's what the filesystem operations module covers.

Questions?